4️⃣

MATERI WEEK 04 - IPC

A. GUIDED PRACTICUM

1. PRACTICUM 4.1 - Pipe

Pipe adalah mekanisme komunikasi satu arah (undirectional) antar proses, dimana satu proses menulis pada 1 sisi pipe, dan proses lainnya membaca dari sisi lain. Ada dua jenis pipe yaitu **Unamed Pipes** & **Named Pipes (FIFO).**

```
Process A (Menulis / Write) ---> { PIPE BUFFER } ---> Process
B (Membaca / Read)
```

a. Unnamed Pipe

```
cat /etc/passwd | cut -f1 -d:

/*
1. Cat (Concatenate) => Membaca isi file dan menampilkan ke s
tandard output
2. | (pipe) => Mengalirkan output dari perintah kiri menjadi
input untuk perintah kanan
3. cut => Mengambil bagian tertentu dari setiap baris teks
4. -f1 => Mengambil field atau kolom pertama
5. -d: => Delimiter :, pembatasnya adalah karakter ":"
*/
```

ini disebut sebagai Unamed Pipe, dimana ciri-cirinya pipe ini tidak dikenal oleh File System.

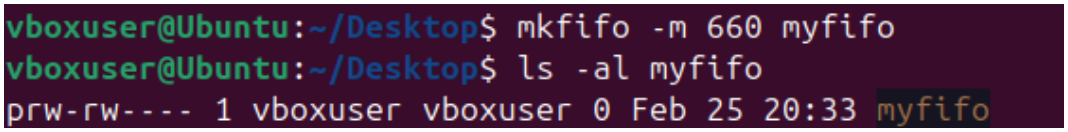
CONTOH :

```
1. ls -l | grep .txt // Filtering Output
2. cut -d: -f1 /etc/passwd | sort // Ambil Username
2. ps aux | grep root | wc -l // Multiple Unnamed Pipes
```

b. Named Pipe (FIFO)

Named pipe adalah mekanisme IPC yang memungkinkan satu atau lebih proses berkomunikasi melalui sebuah saluran / pipe yang memiliki nama dan direpresentasikan di File System.

```
mkfifo -m 660 myfifo
ls -al myfifo
```



Perhatikan tipe file, hak akses, dan ukurannya. Ini adalah named pipe atau FIFO yang merupakan file khusus dalam sistem file Unix / Linux yang dapat digunakan untuk komunikasi antar process (IPC).

Pipe ini bisa diberi nama sehingga bersifat persistent dan dikenali oleh File System, tetapi ukurannya selalu Nol (0 Byte) karena sebenarnya ia hanya buffer di memori

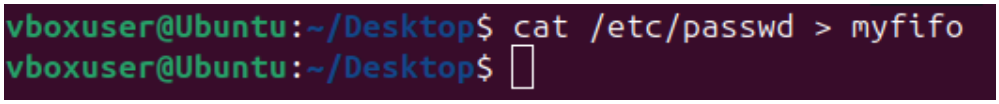
```
prw-rw---- 1 vboxuser vboxuser 0 Feb 25 20:33 myfifo

/*
1. p menunjukan myfifo adalah special file dimana memiliki fl
ag p yang artinya dia pipe
*/
```

CONTOH : Menggunakan myfifo sebagai pipe antara 2 process dalam 2 terminal berbeda

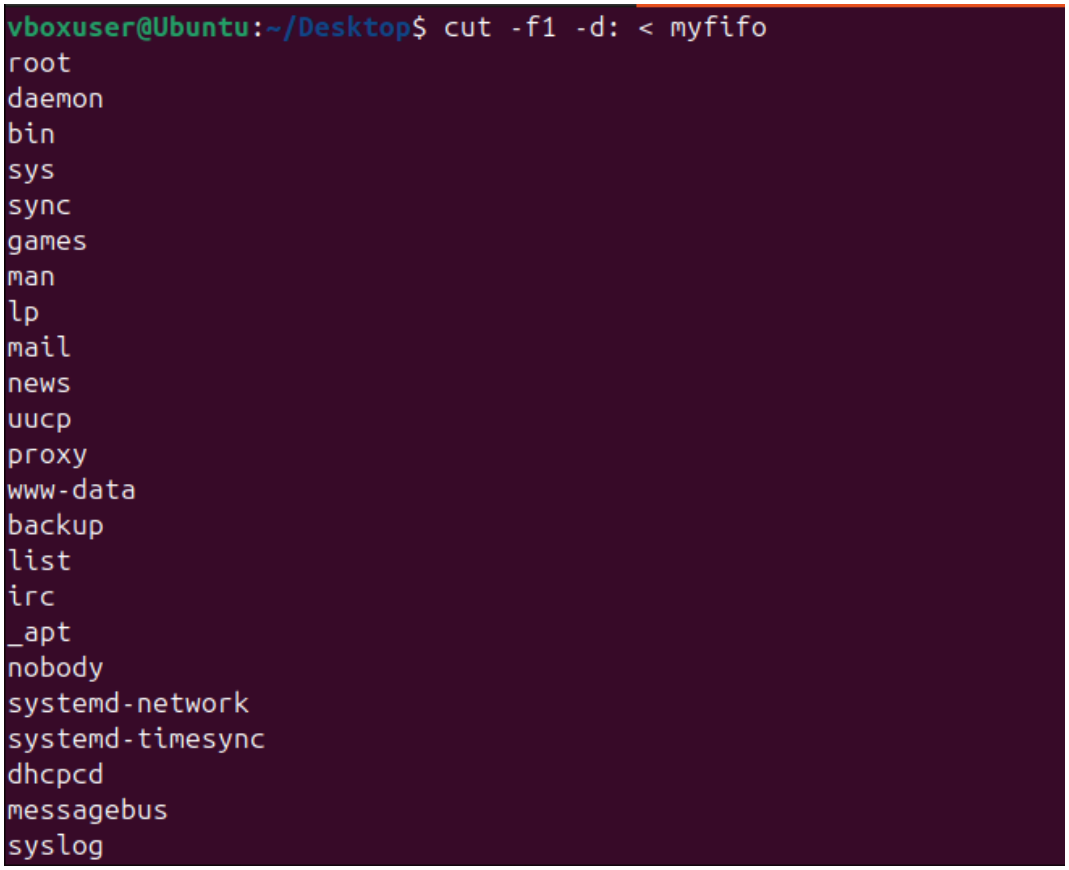
Buka terminal 2 yang berbeda, dan masukkan perintah ini pada masing-masing terminal

```
// Terminal 1 -- Writer
cat /etc/passwd > myfifo

// Terminal 2 -- Reader
cut -f1 -d: < myfifo
```

1. Hasil pada terminal 1



2. Hasil pada terminal 2



Named pipe (FIFO) lebih berguna dibandingkan Unnamed pipes karena mereka mengizinkan 2 process yang unrelated untuk bertukar data.

c. Praktikum 4.1 : 2 File C yang menggunakan named pipe untuk berkomunikasi

1. Buat program C untuk memanggil dan membaca “fifo1” (Reader)

```
// SAVE DENGAN NAMA : rdfifo_NIM.c

#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <limits.h>

int main(void) {
    int fd; // file descriptor untuk fifo
    int len;
    char buf[PIPE_BUF]; // terdefine di /usr/include/linux/li
mits.h
    mode_t mode = 0666; // mode untuk file attributes

    // Membuat named pipe "fifo1", jika berhasil lanjut
    // Jika gagal karena sudah ada -> Lanjut
    // Jika gagal karena belum ada -> program berhenti
    if((mkfifo("fifo1", mode)) < 0) {
        int error = errno;
        if(error == EEXIST) {
            printf("Continue using an existing pipe\n");
        } else {
            perror("mkfifo");
            exit(EXIT_FAILURE);
```

```
        }
    }
    // Open the FIFO read-only
    // BLOCKING => Jika belum ada writer yang membuka FIFO da
lam mode write
    if((fd = open("fifo1", O_RDONLY)) < 0) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    // Read and display the FIFO's output until EOF
    while((len = read(fd,buf,PIPE_BUF-1)) > 0) {
        printf("rdfifo read: %s", buf);
    }
    close(fd);
    remove("fifo1");
    exit(EXIT_SUCCESS);
}
```

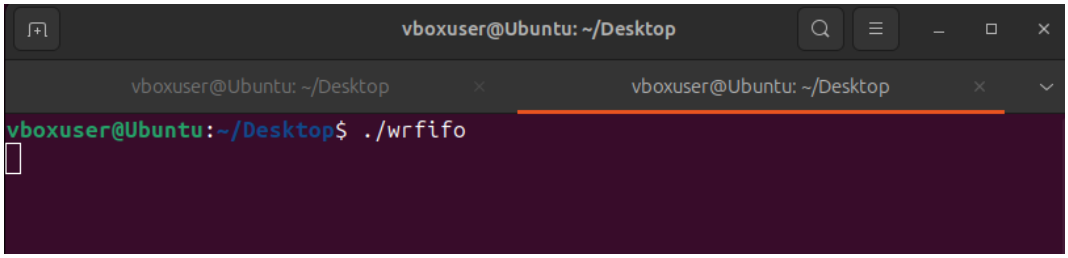
2. Buat program C untuk menulis ke “fifo1” (Writer)

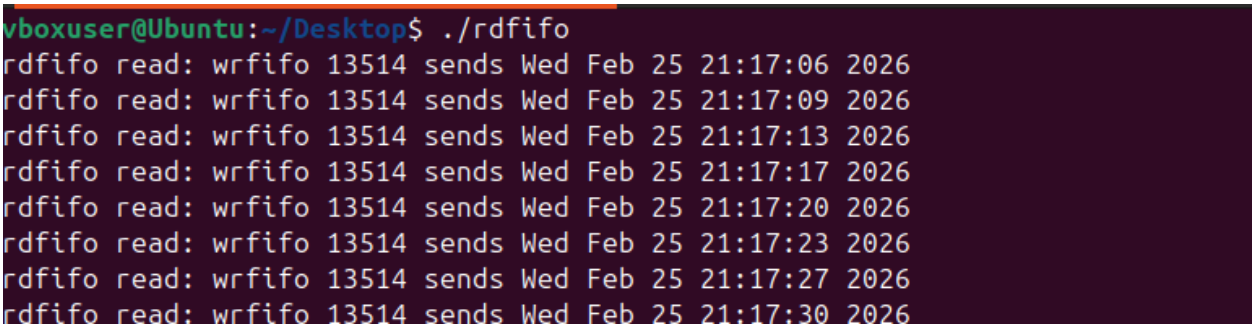
```
// SAVE DENGAN NAMA : wrfifo_NIM.c

#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <limits.h>
#include <time.h>

int main(void)
{
    int fd;
```

```
    int len;
    char buf[PIPE_BUF];
    time_t tp;

    // Open the FIFO write only
    // BLOCKING : Jika belum ada reader yang membuka FIFO
    if ((fd = open("fifo1", O_WRONLY)) < 0)
    {
        perror("open");
        exit(EXIT_FAILURE);
    }

    // generate some data to write
    while (1)
    {
        time(&tp);  // ambil current time -> time.h

        // create the string to write
        len = sprintf(buf, "wrfifo %d sends %s", getpid(), ct
ime(&tp));

                // Menulis ke FIFO
                // Kenapa len + 1 ? => Karena ingin ikut menu
lis karakter Null terminator \0
        if (write(fd, buf, len + 1) < 0)
        {
            perror("write");
            close(fd);
            exit(EXIT_FAILURE);
        }

        sleep(3);
    }
        // Tidak pernah tercapai kecuali program di kill
    close(fd);
```

```
    exit(EXIT_SUCCESS);
}
```

3. Eksekusi masing-masing file C pada terminal yang berbeda
4. Jalankan “rdfifo” terlebih dahulu, baru “wrfifo”





d. Penjelasan Praktikum 4.1

```
Writer Process  --->  Kernel FIFO Buffer  --->  Reader Proces
s
```

Hal ini menjelaskan bahwa memang IPC dengan fifo merupakan :

- One-way communication
- FIFO Ordering
- Blocking I/O
- Buffer dikelola kernel

JIKA SKENARIO-NYA SEPERTI INI :

1. **SKENARIO A : Reader Jalan dulu**

- READER

```
open("fifo1", O_RDONLY) // BLOCK : Karena belum ada writer
```

- WRITER

```
open("fifo1", O_WRONLY) // Kedua open dan selesai
```

Sistem berjalan dengan normal

2. **SKENARIO B : Writer Jalan Dulu**
   - WRITER

```
open("fifo1", O_WRONLY) // BLOCK : Karena belum ada reader
```

   - READER

```
open("fifo1", O_RDONLY) // Open dan selesai
```

Sistem berjalan dengan normal

3. **SKENARIO C : Reader Read Saat Tidak ada Data**, jika FIFO Kosong dan Writer masih hidup, maka akan terjadi Block
4. **SKENARIO D : Writer Write Saat Buffer Penuh**

FIFO punya buffer kernel, misal 64KB (tergantung system), jika Reader Lambat maka Buffer penuh → write() akan BLOCK sampai ada ruang kosong

5. **SKENARIO E : Reader Mati**

Jika reader terminate, Writer saat write() :

- Kernel kirim SIGPIPE
- write() return -1
- errno = EPIPE

Program writer anda akan mengirimkan error.

```
perror("write");
exit(EXIT_FAILURE);
```

6. SKERNARIO F : Writer Mati

Reader saat read() :

- read() return 0 EOF
- Loop Berhenti
- FIFO dihapus

2. PRACTICUM 4.2 - Shared Memory

Shared memory adalah segmen memori yang bisa diakses oleh beberapa process secara bersamaan. Berikut perbedaannya dengan pipe :

Pipe	Shared Memory
Berbasis stream	Berbasis memory
Lambat (copy data)	Sangat cepat
Sederhana	Perlu sinkronisasi

Shared memory adalah sebuah segmen memori yang dapat diakses secara bersamaan oleh beberapa process. Dengan cara ini, process-process dapat berkomunikasi dengan membaca dan menulis pada blok memori yang sama.

Untuk melihat segment shared memory yang tersedia, gunakan perintah:

```
ipcs -m // -m = memory
```

a. REVIEW : Apa itu `argc` dan `argv` ?

Dalam C, function main bisa menerima parameter command-line. Hal ini berarti program dapat menerima input saat dijalankan dari terminal

```
int main(int argc, char *argv[])
```

- argc (Argument Count)

argc → Jumlah argumen yang dikirim dari command-line

CATATAN : argc selalu minimal 1, karena nama program dihitung sebagai argumen pertama

```
Jika, ./shmdemo

Maka :
argc = 1
argv[0] = "./shmdemo"

Jika, ./shmdemo hello

Maka :
argc = 2
argv[0] = "./shmdemo"
argv[1] = "hello"
```

- argv (Argument Vector)

argv adalah array string, dimana

```
argv[0] → nama program
argv[1] → argumen pertama
argv[2] → argumen kedua
...
```

Pada program shmdemo.c dibawah, argc dan argv digunakan pada bagian :

```
if (argc > 2) {
    fprintf(stderr, "usage: ./shmdemo <data string>\n");
    exit(1);
}
```

Artinya :

- Program hanya menerima maksimal 1 argumen tambahan
- Jika lebih dari 1 argumen tambahan → Error

Pada program shmdemo.c juga ada peran argc dalam logika shared-memory :

```
if (argc == 2) {
    printf("write: \"%s\"\n", argv[1]);
    strncpy(data, argv[1], SHM_SIZE);
}
else {
    printf("read : \"%s\"\n", data);
}


Jika Kondisi :
argc == 2 // Mode WRITE
argc == 1 // Mode READ
```

b. shmdemo.c

```
// SAVE DENGAN NAMA : shmdemo_NIM.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h> // IPC KEY
#include <sys/shm.h> // Shared-Memory Library

#define SHM_SIZE 1024  // 1 KB shared memory

int main(int argc, char *argv[])
{
    key_t key;
    int shmid, mode;
```

```
    char *data;

    if (argc > 2) {
        fprintf(stderr, "usage: ./shmdemo <data string>\n");
        exit(1);
    }

    // membuat key
    key = ftok("shmdemo.c", 'R');

    // connect ke segment memory atau create segment memory
    shmid = shmget(key, SHM_SIZE, 0644 | IPC_CREAT);

    // attach ke segment untuk mendapatkan pointer
    data = (char *) shmat(shmid, (void *)0, 0);

    // read atau write berdasarkan command line
    if (argc == 2) {
        printf("write: \"%s\"\n", argv[1]);
        strncpy(data, argv[1], SHM_SIZE);
    }
    else {
        printf("read : \"%s\"\n", data);
    }

    // detach dari segment
    shmdt(data);

    return 0;
}
```

```
vboxuser@Ubuntu: ~/Desktop          vboxuser@Ubuntu: ~/Desktop
vboxuser@Ubuntu:~/Desktop$ ./shmdemo Halo_Operating_System_Kelas_BL
write: "Halo_Operating_System_Kelas_BL"
vboxuser@Ubuntu:~/Desktop$ ./shmdemo
read : "Halo_Operating_System_Kelas_BL"
vboxuser@Ubuntu:~/Desktop$
```

Observasi Shared-Memory Segment yang terbentuk dengan `ipcs -m`

```
vboxuser@Ubuntu:~/Desktop$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x520208d2 0          vboxuser   644        1024       0
```

Kalian dapat menggunakan `ipcrm -m <shmid>` untuk menghapus alokasi memorynya.

```
vboxuser@Ubuntu:~/Desktop$ ipcrm -m 0
vboxuser@Ubuntu:~/Desktop$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
```

c. APA ITU `ftok` ? dan Modifikasi shmdemo.c

```
key_t ftok(const char *pathname, int proj_id);
```

`ftok()` menghasilkan IPC key yang digunakan oleh :

```
shmget()
msgget()
semget()
```

hal ini bertujuan untuk untuk mengidentifikasi resource IPC

- Cara kerja `ftok()` ?

Key akan dibentuk berdasarkan kombinasi :

1. inode file dari pathname
2. Device ID

3. Project ID (karakter kedua)

```
Key = hash(inode + device + proj_id)
```

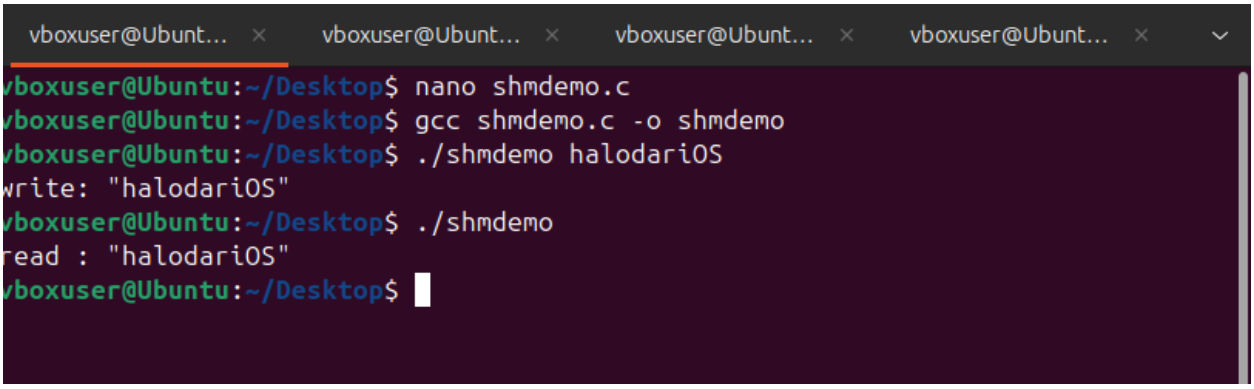
Jika ada yang berubah, maka shared-memory segment yang baru akan terbentuk. Shared memory diindentifikasi berdasarkan key. Jika kedua process menggunakan key yang sama, mereka akan mengakses shared memory yang sama. Jika salah satu menggunakan yang berbeda, mereka tidak lagi berbagi memory

Coba ganti key values, contohnya :

```
// SAVE DENGAN NAMA : shmdemo-modif_NIM.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024  // 1 KB shared memory

int main(int argc, char *argv[])
{
    key_t key;
    int shmid, mode;
    char *data;

    if (argc > 2) {
        fprintf(stderr, "usage: ./shmdemo <data string>\n");
        exit(1);
    }

    // membuat key
    key = ftok("shmdemo.c", 'R');
```

```
        // key = ftok("shmdemo.c", 'B');
        // key = ftok(".", 'A');

    // connect ke segment memory atau create segment memory
    shmid = shmget(key, SHM_SIZE, 0644 | IPC_CREAT);

    // attach ke segment untuk mendapatkan pointer
    data = (char *) shmat(shmid, (void *)0, 0);

    // read atau write berdasarkan command line
    if (argc == 2) {
        printf("write: \"%s\"\n", argv[1]);
        strncpy(data, argv[1], SHM_SIZE);
    }
    else {
        printf("read : \"%s\"\n", data);
    }

    // detach dari segment
    shmdt(data);

    return 0;
}
```

HASILNYA :



```
vboxuser@Ubunt...   vboxuser@Ubunt...   vboxuser@Ubunt...   vboxuser@Ubunt...
vboxuser@Ubuntu:~/Desktop$ nano shmdemo.c
vboxuser@Ubuntu:~/Desktop$ gcc shmdemo.c -o shmdemo1
vboxuser@Ubuntu:~/Desktop$ ./shmdemo1 halodariOS
write: "halodariOS"
vboxuser@Ubuntu:~/Desktop$ ./shmdemo1
read : "halodariOS"
vboxuser@Ubuntu:~/Desktop$
```

```
vboxuser@Ubunt...   vboxuser@Ubunt...   vboxuser@Ubunt...   vboxuser@Ubunt...
vboxuser@Ubuntu:~/Desktop$ nano shmdemo.c
vboxuser@Ubuntu:~/Desktop$ gcc shmdemo.c -o shmdemo2
vboxuser@Ubuntu:~/Desktop$ ./shmdemo2
read : ""
vboxuser@Ubuntu:~/Desktop$ ./shmdemo2 halodariOS
write: "halodariOS"
vboxuser@Ubuntu:~/Desktop$ ./shmdemo2
read : "halodariOS"
vboxuser@Ubuntu:~/Desktop$
```

```
vboxuser@Ubuntu:~/Desktop$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x520208d2 1          vboxuser   644        1024       0
0x410233b1 2          vboxuser   644        1024       0
0x420208d2 3          vboxuser   644        1024       0

vboxuser@Ubuntu:~/Desktop$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x520208d2 1          vboxuser   644        1024       0
0x410233b1 2          vboxuser   644        1024       0
0x420208d2 3          vboxuser   644        1024       0
```

Hapus shared memorynya menggunakan `ipcrm -m <shmid>`

```
vboxuser@Ubuntu:~/Desktop$ ipcrm -m 3
vboxuser@Ubuntu:~/Desktop$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x520208d2 1          vboxuser   644        1024       0
0x410233b1 2          vboxuser   644        1024       0
```

Shared memory segment juga dapat dihapus dengan menggunakan program berikut :

```
// SAVE DENGAN NAMA : shmutil_NIM.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main(int argc, char *argv[])
{
    int shmid;
    // Validasi argumen, dimana program mengharuskan 1 parame
ter tambahan
    // Program akan mengharuskan ./shmutil_modif.c <shmid>
    if (argc != 2) {
        fprintf(stderr, "usage: %s <shmid>\n", argv[0]);
        exit(1);
    }
        // Konversi shmid menjadi integer
    shmid = atoi(argv[1]);

        // Hapus shared memory
    shmctl(shmid, IPC_RMID, (struct shmid_ds *)0);
```

```
    return 0;
}
```

HASILNYA :

```
vboxuser@Ubuntu:~/Desktop$ nano shmutil.c
vboxuser@Ubuntu:~/Desktop$ gcc shmutil.c -o shmutil
vboxuser@Ubuntu:~/Desktop$ ./shmutil 1
vboxuser@Ubuntu:~/Desktop$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x410233b1 2          vboxuser   644        1024       0
```

Program ini adalah utility untuk menghapus shared-memory segment pada sistem IPC. Berikut perbedaan `shmctl` dan `shmdt()`

Aspek	`shmdt()`	`shmctl()`
Fungsi utama	Detach shared memory dari proses	Kontrol / manajemen shared memory
Scope	Per proses	Global (segment level)
Menghapus segment?	Tidak	Bisa (IPC_RMID)
Mengubah metadata?	Tidak	Bisa
Dipanggil oleh	Proses yang attach	Biasanya admin / owner

3. PRACTICUM 4.3 - Race Condition

Pada bagian praktikum ini, kita akan menggunakan Shared-Memory Segment sebagai tempat menyimpan data dan dapat di modifikasi oleh parent dan child process yang berjalan bersamaan.

```
// SAVE DENGAN NAMA : shmrace_NIM.c

#include <stdio.h>
#include <stdlib.h>
```

```
#include <sys/types.h>
#include <sys/shm.h>
#include <sys/ipc.h>
#include <unistd.h>
#include <sys/wait.h>

#define NLOOPS 100000

int main()
{
    int i, child, shmid, key = 1000, *ptrData;
        // Membuat shared-memory
    shmid = shmget(key, sizeof(int), 0644 | IPC_CREAT);
    // Attach ke process
    ptrData = (int *) shmat(shmid, (void *)0, 0);

        // Inisialisasi data
    *ptrData = 0;


        // Fork process
    child = fork();

    // RACE CONDTION TERJADI DARI SINI
    // Terjadi karena parent dan child berlomba-lomba increme
nt bersama
    if (child == 0)
    {
        for (i = 0; i < NLOOPS; i++)
        {
            (*ptrData)++;
            printf("i=%d child: ptrData's value=%d\n", i, *pt
rData);
        }
    }
    else if (child > 0)
```

```
    {
        for (i = 0; i < NLOOPS; i++)
        {
            (*ptrData)++;
            printf("i=%d parent: ptrData's value=%d\n", i, *p
trData);
        }

        int status;
        waitpid(child, &status, 0);
    }

        // DETACH shared memory dari process
    shmdt(ptrData);
    // HAPUS shared memory dari kernel
    shmctl(shmid, IPC_RMID, (struct shmid_ds *)0);

    return 0;
}
```

JAWAB PERTANYAAN INI DENGAN KATA-KATA KALIAN, SAVE DENGAN nama "jawaban_NIM.txt" !

1. WHAT DO YOU OBSERVE ?

Ada beberapa hal yang dapat kita amati ketika program berjalan yaitu :

- Output PARENT PROCESS dan CHILD PROCESS akan INTERLEAVING (Saling Menimpa dan Tumpang Tindih)
- Nilai `ptrData` naik tidak berurutan
- Nilai akhir sering tidak sama setiap kali program dijalankan.
- Kadang lebih kecil dari nilai yang seharusnya ≤ 200000

2. DID YOU GET CONSISTENT RESULT FOR ptrData's VALUE ?

Tidak karena nilai akhir `ptrData` biasanya berbeda-beda setiap kali eksekusi. Hampir selalu kurang dari nilai teoritis. Hal ini terjadi karena program bersifat non-

deterministic

3. WHAT SHOULD THE CORRECT VALUE ?

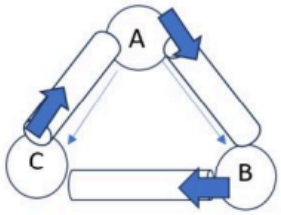
Secara matematis adalah NLOOPS ∗ 2, mengapa dikali 2 karena ada 2 Process yaitu PARENT dan CHILD

4. WHY DID YOU GET INCONSISTENT RESULT ?

Race Condition → Jelaskan mengapa race condition dari alur yang sudah ada.

4. PRACTICE

Given the following process hierarchy, construct an inter-process communication mechanism using named pipes (FIFO) between the following processes, such that each time process A prints a string, it also receives a reply from the string sent through three named pipes, as shown in the figure, and displays it on the screen as an echo.



IDE UTAMANYA : SIKLUS dan REMOVE BLOCKING pada `open()`

Pada FIFO (named pipe) :

1. open(....., O_WRONLY) akan block jika belum ada reader
2. open(....., O_RDONLY) akan block jika belum ada writer

Kedua hal ini mengakibatkan ketika A jalan dulu, A akan hang / menunggu terus menerus. Sehingga urutan eksekusi menjadi penting.

Ini bukan karena kesalahan logika IPC, tetapi karena mekanisme sinkronisasi bawaan FIFO di kernel.

SOLUSINYA menggunakan O_RDWR yang merupakan gabungan dari : Reader + Writer dalam satu proses sehingga tidak perlu menunggu

- A → B
- B → C

- C → A

CATATAN :

Setiap kali A mengirim string, string tersebut : Berarti ada 3 Program .c (Beri nama ProcessA_NIM.c, ProcessB_NIM.c, dan ProcessC_NIM.c) !

- Dikirim ke B
- B meneruskan ke C
- C meneruskan ke A
- A menampilkan hasil sebagai ECHO

  👷 SOLUTION - WEEK04 - IPC